

COMS W4156 Advanced Software Engineering (ASE)

October 12, 2023



Agenda

1. mocking demo: Ansh Kothary
2. integration testing



mocking demo: Ansh Kothary

https://github.com/ank2809/mocking_demo_4156

Agenda

1. mocking demo: Ansh Kothary
2. integration testing

$$Life = \int_{birth}^{death} \frac{happiness}{time} \Delta time$$

What is Integration Testing?

The goal of [integration testing](#) is to find errors at the interfaces **between** components (classes or other modular units) and assemblies consisting of multiple components. Thus a special case of greybox testing

Integration testing considers two or more components that communicate or depend on each other in some way (or where one component communicates with or depends on another component, does not need to be reciprocal)

Integration tests are often automated via the same testing tools as unit testing, with constructed inputs, assertions that check outputs, setup, teardown, etc.

	Unit Testing	Integration Testing
		
		



Integration Testing with Test Doubles

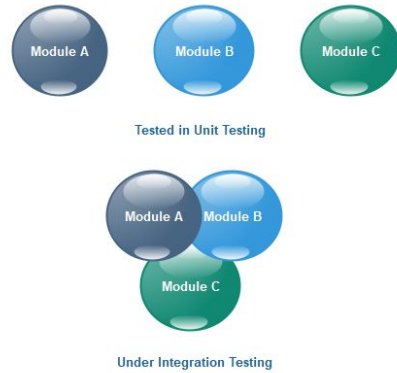
Like unit and class testing, integration testing should fake dependencies outside the code under test

When an integration test fails, we want to know the bug is in one or more of the components being integrated into the assembly, or at the interface being integrated, not some other part of the system

However, integration testing eventually needs to test interfaces with external resources - in which case real resources are used after initial testing with test doubles

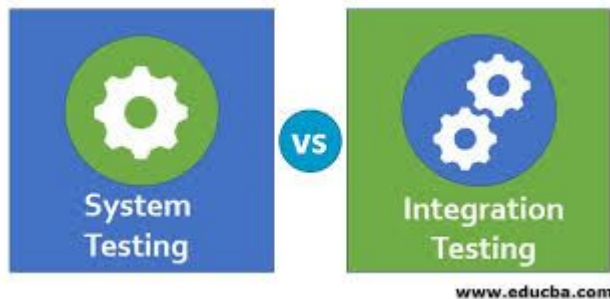


Choosing Integration Testing Inputs



Similar to class testing except 1. some integration order strategy is usually chosen in advance and 2. source code may not be available for all components to be integrated (e.g., third-party libraries, external resources)

Given components (or previously integrated assemblies of components): reuse their unit/class testing inputs (e.g., from equivalence partitioning + boundary analysis), replace test doubles with real code from the other components being integrated, choose additional test inputs to force reaching replaced test doubles as needed



Example

```
myClass foo (myClass obj, something x) {  
    local anotherThing y;  
  
    some code;  
    if (something rarely true about x) {  
        y = methodInSameClass(obj,x);  
    }  
    some more code;  
    if (something often true about x) {  
        obj.field = methodOutsideClass(y);  
    }  
    code that does something;  
  
    return obj;  
}
```

? What test inputs do we need for unit testing ?

? What do we do differently for class testing ?

? What do we do differently for integration testing ?



Integration Bugs



A common cause of integration bugs is inconsistencies between the viewpoints of different modules or units

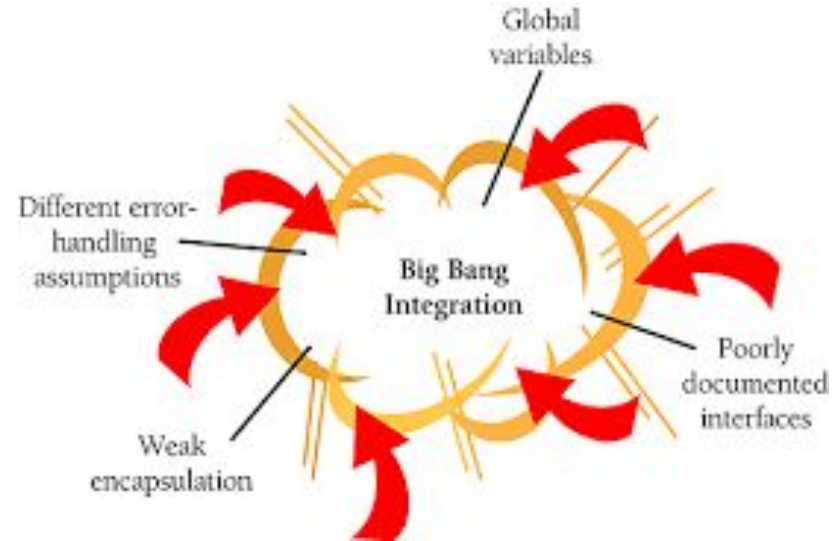
The parts work fine in isolation but do not work together

This may indicate a problem with test doubles reflecting the point of view of the unit under test and not the unit they are standing in for

? Why might this happen ?

Integration Order Strategies

“Big bang” approach to integration testing: Test all independently developed components in isolation, then test the whole system. All done!



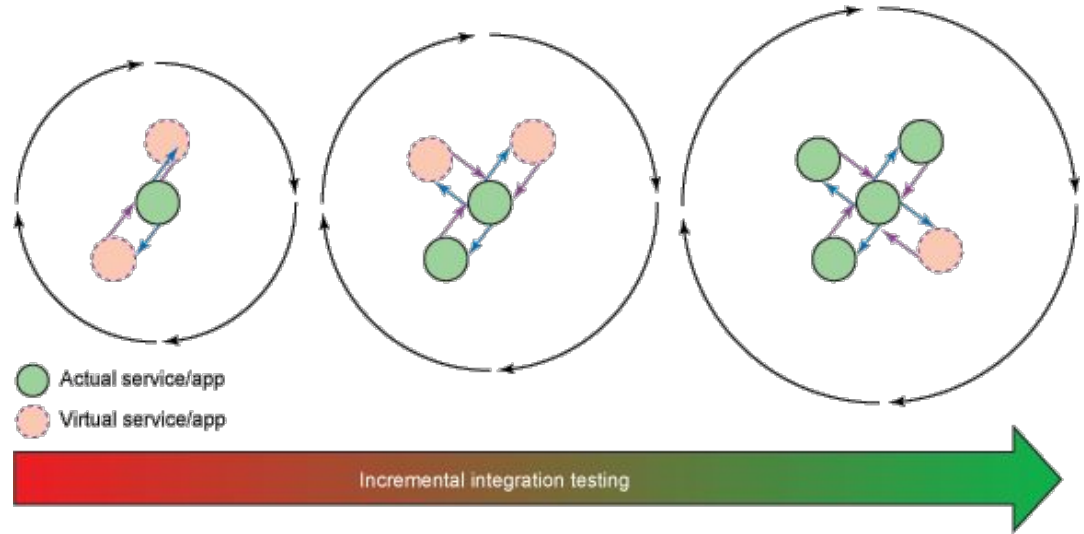
Not so fast: Bugs detected during “big bang” could be anywhere!

Imagine Big Bang Testing for Automobiles



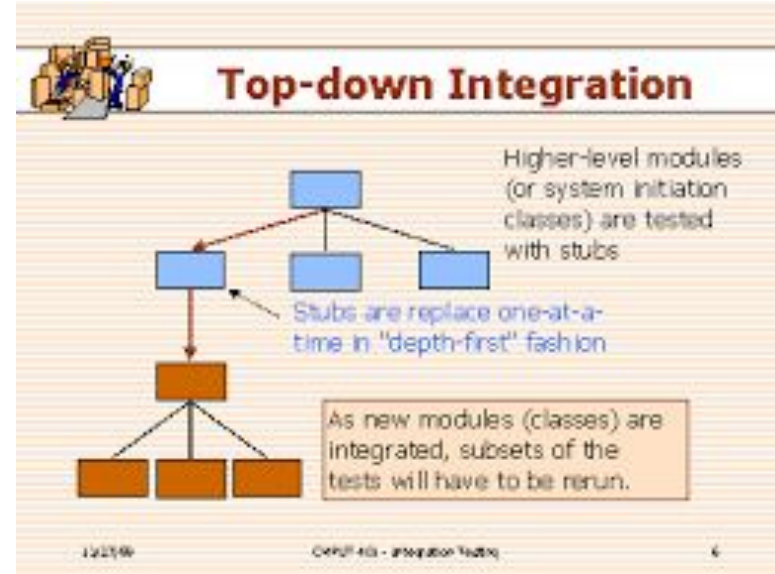
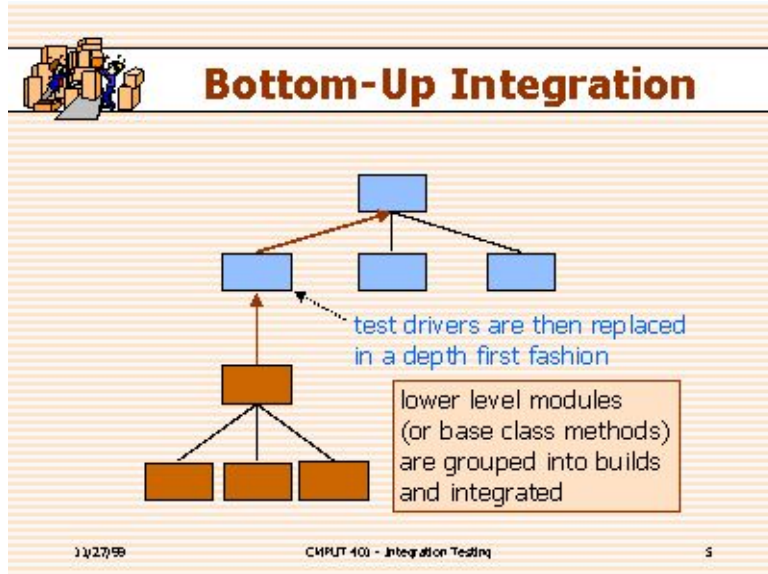
Incremental Integration Helps Localize Bugs

At each step, pick two components (or assemblies of components) that have both already been tested individually and at least one is dependent in some way on the other. Continue using test doubles for all other components



Since only a small number of components (ideally two) are integrated at a time, that limits where we need to look to find the “root cause” (the underlying coding mistake or interface mismatch) and fix the bug

Traditional Integration Testing Orders



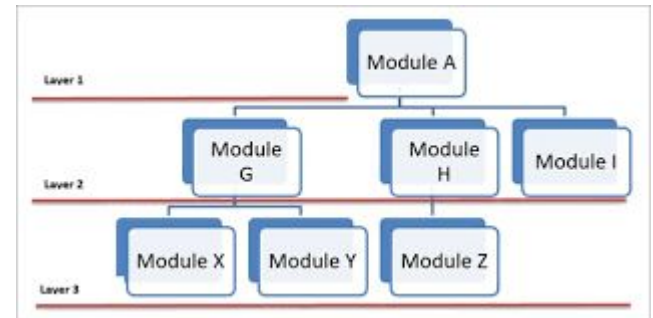
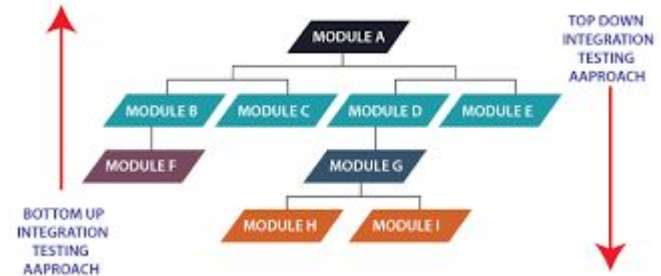
Bottom up starts integration process from lowest layer, top down starts integration process from top

Integration Testing Order is not Test Runner Order

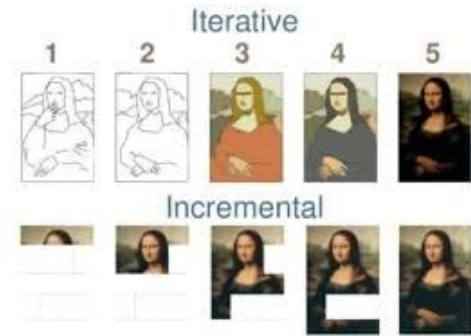
Developers **add to** test suite incrementally, integrating components and assemblies of components in some order

This order refers to the order of writing new integration tests and adding them to the test suite

Orthogonal to test runner order, which determines the order of running tests in a given test suite



Top Down vs. Bottom Up



TOP DOWN INTEGRATION TESTING	BOTTOM UP INTEGRATION TESTING
Top Down Integration testing is an approach of Integration testing in which integration takes place from top to bottom, meaning system integration begins with top level modules.	Bottom Up Integration testing is an approach to Integration testing in which integration takes place from bottom to top, meaning system integration begins with lowest level modules.
The higher level modules are tested first, then the lower level modules are tested, and then the modules are integrated accordingly.	The lower level modules are tested first, then the higher level modules are tested, and then the modules are integrated accordingly.
Stubs/mocks are used to simulate a submodule if the invoked submodule is not developed or integrated yet, working as a temporary replacement.	Drivers are used to simulate the main module if the main module is not developed or integrated yet, working as a temporary replacement.

What's the Difference between Top and Bottom?

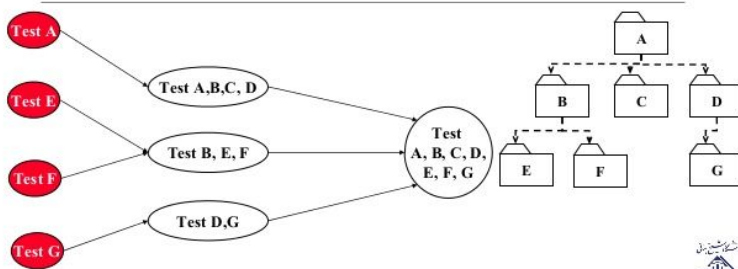


TOP DOWN INTEGRATION TESTING	BOTTOM UP INTEGRATION TESTING
Most beneficial if significant defects occur toward the top of the program.	Most beneficial if crucial flaws are encountered towards the bottom of the program.
The high-level modules are designed/created/tested first, then lower-level submodules/subroutines are called from them.	Low-level modules are designed/created/tested first, then these submodules/subroutines are called from the high-level modules.
Commonly implemented in procedure-oriented programming languages.	Commonly implemented in object-oriented programming languages.
The complexity of this testing is simple.	The complexity of this testing is complex and highly data intensive.
Works on big to small components.	Works on small to big components.
Stub/mock modules must be produced.	Driver modules must be produced.

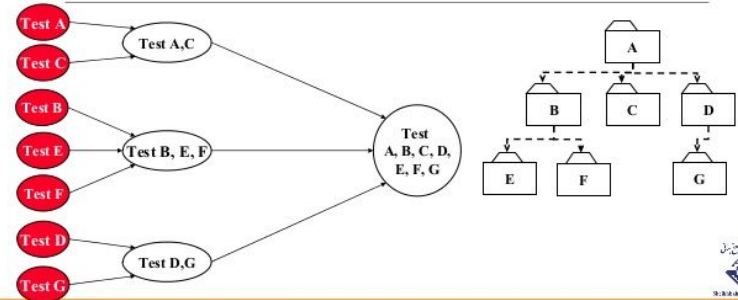
Are There Other Options?

Hybrid (Sandwich) Integration Testing integrates “natural” assemblies or layers, meet in middle

Sandwich Testing Strategy

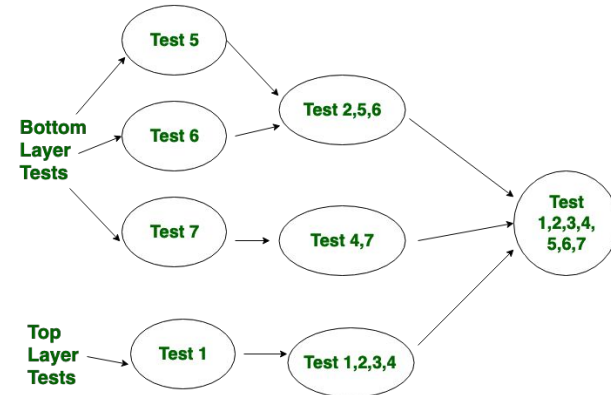
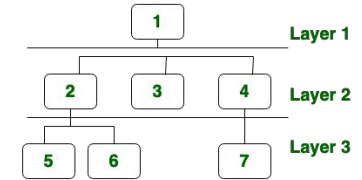


Modified Sandwich Testing



Hybrid Integration Testing

- Combines both top down and bottom up strategies.
- Basically viewed as 3 layers:
 - (i) Main target layer
 - (ii) A layer above the target layer
 - (iii) A layer below the target layer
- Testing mainly focused on main target layer, selected on the basis of system characteristics and code structures.
- Tries to minimize the number of stubs and drivers when there are more than 3 layers.



Sandwich Integration Testing

Advantages:

- Used in very large projects with sub-projects.
- Allows parallel testing.
- Time saving approach.
- Achieves more coverage with same stubs.

Disadvantages:

- More costly.
- Cannot be used for systems that have a lot of interdependencies between different modules.
- Needs more stubs and drivers.



Other Integration Testing Strategies

History: Prioritize testing interfaces where the most bugs, or the most challenging bugs, have been found previously

Risk: Prioritize testing the *most risky* interfaces, e.g., a new component being introduced to an existing system, replacing an existing third-party component with an alternative component from a different third party, replacing an existing third-party component with a new version of that component

Expense: Prioritize testing the interfaces where debugging will be the most difficult and/or expensive



Upcoming Assignments

- [First Iteration](#) due Thursday, October 19, by 11:59pm
- [First Iteration Demo](#) due Thursday, October 26, by 11:59pm



Next Week

logging and log analysis

continuous integration



Ask Me Anything